Filter 실행 계획 상세 분석 (Filter Execution Plan)

파일 개요

- **파일 경로**: `crates/proof-of-sql/src/sql/proof_plans/filter_exec.rs`
- **주요 역할**: SQL WHERE 절을 처리하는 필터링 로직 구현
- **핵심 기능**: Boolean 마스크를 이용한 영지식 필터링 증명

모듈 구조 및 의존성

주요 Import 분석

```
use crate::{
base::{
database::{
filter_util::filter_columns, Column, ColumnField, ColumnRef, LiteralValue, Owned
Table, TableEvaluation, TableOptions, TableRef,
},
// ... 기타 모듈들
},
sql::{
proof::{
```

```
FinalRoundBuilder, FirstRoundBuilder, HonestProver, ProofPlan, ProverEvaluate,
ProverHonestyMarker, VerificationBuilder,
},
proof_exprs::{AliasedDynProofExpr, DynProofExpr, ProofExpr, TableExpr},
proof_gadgets::{final_round_evaluate_filter, verify_evaluate_filter},
},
};
```

설명:

- **filter_util::filter_columns**: 실제 필터링 연산을 수행하는 유틸리티
- **proof_gadgets**: 필터링에 대한 영지식 증명 로직
- **ProofPlan**: 증명 가능한 실행 계획의 기본 인터페이스

핵심 데이터 구조

1. OstensibleFilterExec 구조체

```rust
#[derive(Debug, PartialEq, Serialize, Deserialize, Clone)]
pub struct OstensibleFilterExec<H: ProverHonestyMarker> {
aliased_results: Vec<AliasedDynProofExpr>,
table: TableExpr,
/// TODO: add docs
where_clause: DynProofExpr,
phantom: PhantomData<H>,
```

```
}
```

설명:

- **aliased_results**: SELECT 절의 결과 표현식들 (별칭 포함)
- **table**: 필터링할 테이블 표현식
- **where_clause**: WHERE 절의 조건 표현식 (Boolean 결과 생성)
- **phantom**: 증명자 정직성 마커 (컴파일 타임 타입 안전성)

2. FilterExec 타입 별칭

```
/// Alias for a filter expression with a honest prover.
pub type FilterExec = OstensibleFilterExec<HonestProver>;
```

설명: 정직한 증명자를 사용하는 필터 실행 계획의 편의 타입입니다.

증명 생성 과정 (ProverEvaluate 구현)

1. 첫 번째 라운드 평가 (first_round_evaluate)

1.1 초기 설정 및 테이블 로드

```
fn first_round_evaluate<'a, S: Scalar>(
&self,
builder: &mut FirstRoundBuilder<'a, S>,
alloc: &'a Bump,
table_map: &IndexMap<TableRef, Table<'a, S>>,
```

```
params: &[LiteralValue],
) -> PlaceholderResult<Table<'a, S>> {
log::log_memory_usage("Start");


let table = table_map
.get(&self.table.table_ref)
.expect("Table not found");
```

설명:

- 메모리 사용량을 로깅하고 필터링할 테이블을 가져옵니다
- Bump 할당자를 사용하여 효율적인 메모리 관리를 수행합니다

1.2 WHERE 절 평가 및 Boolean 마스크 생성

```
// 1. selection
let selection_column: Column<'a, S> = self
.where_clause
.first_round_evaluate(alloc, table, params)?;
let selection = selection_column
.as_boolean()
.expect("selection is not boolean");
let output_length = selection.iter().filter(|b| **b).count();
```

설명:

- **핵심 단계**: WHERE 절을 평가하여 Boolean 마스크를 생성합니다
- `selection` 은 각 행이 조건을 만족하는지를 나타내는 Boolean 배열입니다
- `output_length` 는 조건을 만족하는 행의 개수입니다

1.3 SELECT 절 컬럼들 평가

```
// 2. columns
let columns: Vec<_> = self
.aliased_results
.iter()
.map(|aliased_expr| -> PlaceholderResult<Column<'a, S>> {
aliased_expr.expr.first_round_evaluate(alloc, table, params)
})
.collect::<PlaceholderResult<Vec<_>>>()?;
```

설명:

- SELECT 절의 모든 표현식을 평가하여 컬럼 데이터를 생성합니다
- 각 표현식은 독립적으로 평가되어 Column 객체로 변환됩니다

1.4 필터링 적용 및 MLE 생성

```
// Compute filtered_columns and indexes
let (filtered_columns, _) = filter_columns(alloc, &columns, selection);
// 3. Produce MLEs
filtered_columns.iter().copied().for_each(|column| {
builder.produce_intermediate_mle(column);
```

```
});
```

설명:

- **filter_columns**: Boolean 마스크를 사용하여 실제 필터링을 수행합니다
- 필터링된 컬럼들을 다선형 확장(MLE)으로 변환하여 증명에 사용합니다
- 이 MLE들은 나중에 sumcheck 프로토콜에서 사용됩니다

1.5 결과 테이블 생성 및 챌린지 요청

```
let res = Table::<'a, S>::try_from_iter_with_options(
self.aliased_results
.iter()
.map(|expr| expr.alias.clone())
.zip(filtered_columns),
TableOptions::new(Some(output_length)),
)
.expect("Failed to create table from iterator");
builder.request_post_result_challenges(2);
builder.produce_chi_evaluation_length(output_length);
```

설명:

- 필터링된 컬럼들로 결과 테이블을 구성합니다
- **2개의 챌린지 요청**: α(alpha)와 β(beta) - 나중에 folding에 사용
- χ(chi) 평가 길이를 출력 길이로 설정합니다

2. 최종 라운드 평가 (final_round_evaluate)

2.1 챌린지 소비 및 재평가

```
fn final_round_evaluate<'a, S: Scalar>(
&self,
builder: &mut FinalRoundBuilder<'a, S>,
alloc: &'a Bump,
table_map: &IndexMap<TableRef, Table<'a, S>>,
params: &[LiteralValue],
) -> PlaceholderResult<Table<'a, S>> {
log::log_memory_usage("Start");
let alpha = builder.consume_post_result_challenge();
let beta = builder.consume_post_result_challenge();
```

설명:

- 첫 번째 라운드에서 요청한 2개의 챌린지를 소비합니다
- 이 챌린지들은 컬럼들을 folding하는 데 사용됩니다

2.2 동일한 평가 과정 반복

```
// 1. selection
let selection_column: Column<'a, S> = self
.where_clause
.final_round_evaluate(builder, alloc, table, params)?;
```

```
let selection = selection_column
.as_boolean()
.expect("selection is not boolean");


// 2. columns
let columns: Vec<_> = self
.aliased_results
.iter()
.map(|aliased_expr| -> PlaceholderResult<Column<'a, S>> {
aliased_expr
.expr
.final_round_evaluate(builder, alloc, table, params)
})
.collect::<PlaceholderResult<Vec<_>>>()?;
```

설명:

- 첫 번째 라운드와 동일한 평가를 수행하지만, 이번에는 FinalRoundBuilder를 사용합니다
- 이는 일관성을 보장하고 sumcheck에 필요한 다항식을 구성하기 위함입니다

2.3 필터 증명 생성

```
final_round_evaluate_filter::<S>(
builder,
alloc,
alpha,
```

```
beta,
&columns,
selection,
&filtered_columns,
table.num_rows(),
result_len,
);
```

설명:

- **핵심 증명 생성**: 필터링이 올바르게 수행되었음을 증명하는 다항식 제약 조건들을 생성합니다
- α와 β 챌린지를 사용하여 컬럼들을 folding합니다

검증 과정 (verifier_evaluate)

1. 챌린지 소비 및 평가 수행

```
fn verifier_evaluate<S: Scalar>(
&self,
builder: &mut impl VerificationBuilder<S>,
accessor: &IndexMap<TableRef, IndexMap<Ident, S>>,
_result: Option<&OwnedTable<S>>,
chi_eval_map: &IndexMap<TableRef, (S, usize)>,
params: &[LiteralValue],
) -> Result<TableEvaluation<S>, ProofError> {
```

```
let alpha = builder.try_consume_post_result_challenge()?;

let beta = builder.try_consume_post_result_challenge()?;
```

설명:

- 검증자도 동일한 α, β 챌린지를 소비합니다
- 이는 증명자와 검증자 간의 동기화를 보장합니다

2. 표현식 검증 평가

```
let input_chi_eval = *chi_eval_map

.get(&self.table.table_ref)

.expect("Chi eval not found");

let accessor = accessor

.get(&self.table.table_ref)

.cloned()

.unwrap_or_else(|| [].into_iter().collect());

// 1. selection

let selection_eval =

self.where_clause

.verifier_evaluate(builder, &accessor, input_chi_eval.0, params)?;

// 2. columns

let columns_evals = Vec::from_iter(

self.aliased_results

.iter()
```

```
.map(|aliased_expr| {
aliased_expr.expr.verifier_evaluate(
builder,
&accessor,
input_chi_eval.0,
params,
)
})
.collect::<Result<Vec<_>, _>>()?,
);
```

설명:

- 검증자는 실제 데이터 대신 commitment된 값들과 평가점에서의 값들을 사용합니다
- WHERE 절과 SELECT 절 표현식들을 평가하여 기대값을 계산합니다

3. 필터 증명 검증

```
let filtered_columns_evals =
builder.try_consume_first_round_mle_evaluations(self.aliased_results.len())?;

let output_chi_eval = builder.try_consume_chi_evaluation()?;

let c_fold_eval = alpha * fold_vals(beta, &columns_evals);
let d_fold_eval = alpha * fold_vals(beta, &filtered_columns_evals);
```

```
verify_evaluate_filter(
builder,
c_fold_eval,
d_fold_eval,
input_chi_eval.0,
output_chi_eval.0,
selection_eval,
)?;
```

설명:

- **c_fold_eval**: 원본 컬럼들의 folded 평가값
- **d_fold_eval**: 필터링된 컬럼들의 folded 평가값
- `verify_evaluate_filter` 를 호출하여 필터링 증명을 검증합니다

필터 증명 메커니즘 (proof_gadgets/filter_base.rs)

1. 증명 제약 조건들

```
pub(crate) fn verify_evaluate_filter<S: Scalar>(
builder: &mut impl VerificationBuilder<S>,
c_fold_eval: S,
d_fold_eval: S,
chi_n_eval: S,
```

```
chi_m_eval: S,

s_eval: S,

) -> Result<(), ProofError> {

let c_star_eval = builder.try_consume_final_round_mle_evaluation()?;

let d_star_eval = builder.try_consume_final_round_mle_evaluation()?;
```

설명:

- **c_star, d_star**: 보조 다항식들 (역원 계산 관련)
- **chi_n, chi_m**: 입력과 출력 길이에 대한 indicator 다항식들
- **s_eval**: selection 마스크의 평가값

2. 제약 조건 1: c_star + c_fold * c_star - chi_n = 0

```
builder.try_produce_sumcheck_subpolynomial_evaluation(

SumcheckSubpolynomialType::Identity,

c_star_eval + c_fold_eval * c_star_eval - chi_n_eval,

2,

)?;
```

설명:

- 이 제약 조건은 c_star = 1/(1 + c_fold)임을 보장합니다
- 즉, c_star * (1 + c_fold) = chi_n (모든 위치에서 1)

3. 제약 조건 2: d_star + d_fold * d_star - chi_m = 0

```
builder.try_produce_sumcheck_subpolynomial_evaluation(
SumcheckSubpolynomialType::Identity,
d_star_eval + d_fold_eval * d_star_eval - chi_m_eval,
2,
)?;
```

설명: 마찬가지로 d_star = 1/(1 + d_fold)임을 보장합니다.

4. 제약 조건 3: sum c_star * s - d_star = 0

```
builder.try_produce_sumcheck_subpolynomial_evaluation(
SumcheckSubpolynomialType::ZeroSum,
c_star_eval * s_eval - d_star_eval,
2,
)?;
```

설명:

- **핵심 제약 조건**: 필터링의 정확성을 보장합니다
- selection이 true인 위치에서만 값이 복사되었음을 증명합니다

5. 제약 조건 4: d_fold * chi_m - d_fold = 0

```
builder.try_produce_sumcheck_subpolynomial_evaluation(
SumcheckSubpolynomialType::Identity,
d_fold_eval * (chi_m_eval - S::ONE),
```

```
2,

)?;
```

설명:

- d_fold가 출력 범위에서만 정의됨을 보장합니다
- chi_m = 1이므로 이 제약 조건은 d_fold * 0 = 0이 됩니다

Boolean 마스크를 이용한 증명 방식

1. 필터링 과정

1. **Boolean 마스크 생성**: WHERE 절 평가로 각 행에 대한 true/false 값 생성
2. **선택적 복사**: Boolean 마스크가 true인 행들만 결과에 포함
3. **압축**: 필터링된 결과는 연속된 메모리 공간에 저장

2. 영지식 증명 방식

1. **Folding**: 여러 컬럼을 하나의 다항식으로 결합 (α, β 챌린지 사용)
2. **역원 계산**: c_star, d_star를 통해 나눗셈 연산을 곱셈으로 변환
3. **제약 조건**: 4개의 다항식 제약 조건으로 필터링 정확성 증명
4. **Sumcheck**: 모든 제약 조건을 sumcheck 프로토콜로 검증

3. 보안성 보장

- **완전성**: 정직한 증명자는 항상 유효한 증명을 생성할 수 있습니다
- **건전성**: 부정직한 증명자는 잘못된 필터링 결과에 대해 유효한 증명을 생성할 수 없습니다

- **영지식성**: 검증자는 실제 데이터 값을 알 수 없고, 오직 필터링이 올바르게 수행되었다는 사실만 확인할 수 있습니다

이 시스템을 통해 SQL WHERE 절의 복잡한 조건들도 효율적으로 영지식 증명할 수 있으며, 대용량 데이터베이스에서도 확장 가능한 성능을 제공합니다.